Discipline : Enseignement de spécialité NSI

Contenus Graphes : structures relationnelles. Sommets, arcs, arêtes, graphes orientés ou non orientés.

Algorithmes sur les graphes.

Capacités attendues : Modéliser des situations sous forme de graphes. Écrire les implémentations correspondantes d'un graphe : matrice d'adjacence, liste de successeurs/de prédécesseurs. Passer d'une représentation à une autre.

Parcourir un graphe en profondeur d'abord, en largeur d'abord. Repérer la présence d'un cycle dans un graphe. Chercher un chemin dans un graphe.

Pré-requis : Les arbres

Vous ferez une copie d'écran de <u>vos résultats</u> pour toutes les questions du TP.

A Recherche du chemin le plus court : Algorithme de Dijkstra

Dans un graphe orienté ou non, chaque arrête possède un poids. Pour atteindre la destination, le chemin avec la somme des poids la plus faible correspond au plus court chemin. L'algorithme calcule les plus courts chemins à partir d'une source vers tous les autres sommets dans un graphe orienté pondéré par des réels positifs en utilisant un parcours en largeur.
L'algorithme porte le nom de son inventeur, l'informaticien néerlandais Edsger Dijkstra, et a été publié en 1959.
Applications :

- Calcul des itinéraires routiers : Le poids des arcs pouvant être la distance (pour le trajet le plus court), le temps estimé (pour le trajet le plus rapide), la consommation de carburant et le prix des péages (pour le trajet le plus économique).
- Une application courante de l'algorithme de Dijkstra apparaît dans les protocoles de routage interne « à état de liens », tels que Open Shortest Path First (OSPF)

Au départ, on considère que les distances de chaque sommet au sommet de départ sont infinies, sauf pour le sommet de départ pour lequel la distance est nulle. Le sous-graphe de départ est l'ensemble vide.

Au cours de chaque itération, on choisit en dehors du sous-graphe un sommet de distance minimale et on l'ajoute au sous-graphe. Ensuite, on met à jour les distances des sommets voisins de celui ajouté. La mise à jour s'opère comme suit : la nouvelle distance du sommet voisin est le minimum entre la distance existante et celle obtenue en ajoutant le poids de l'arc entre sommet voisin et sommet ajouté à la distance du sommet ajouté.

B (/10pts) Travail sur l'algorithme

Le pseudo-code suivant donne l'architecture de l'algorithme :

```
Fonction Dijkstra(mat_adj,sommet_depart, sommet_arrivee)
        observe = créer file fifo
        visite=liste_vide
        chemin=liste_vide
        dejavu=liste_vide
        enfiler sommet_depart à observe
        distance= {sommet_depart :0}
```

```
# création d'un dictionnaire avec la clé nom du sommet et valeur de distance ici 0 car c'est le départ
tant que observe n'est pas vide
        sommet recoit le premier élément de observe (defiler)
        pour tous les voisins de mat_adj[sommet]  (successeurs)
                ajouter voisin à observe
                nouvelle distance = distance[sommet]+ mat_adj[sommet][voisin]
                si voisin n'est pas dans distance ou nouvelle distance< distance[voisin] :
                        distance[voisin]=nouvelle distance
                        ajouter voisin à deja_vu
                        ajouter sommet à visite
ajouter sommet_arrivee à chemin

pour variable i de (taille de dejavu) à 0
        Si dernière valeur de chemin est égal à dejavu indice i
                Ajouter visite indice i à chemin
Si sommet de départ n'est pas dans chemin
        Ajouter sommet_depart à chemin
Inverser la liste chemin
retourner chemin, distance[sommet_arrivee]
```

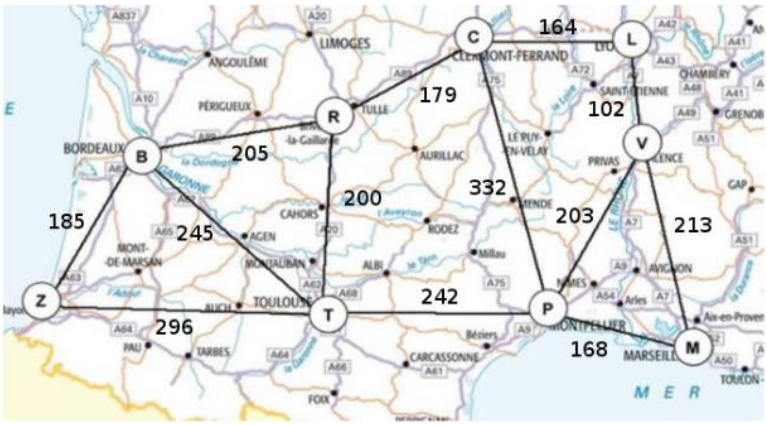
B-1 (/5pts) Créer la classe graphe et rechercher l'algorithme en l'intégrant comme méthode de la classe. Tester votre programme pour la liste d'adjacence des successeurs suivante :

adj = {'A':{'B':15,'C':4},'B':{'E':5},'C':{'E':11,'D':2},'D':{'E':3},'E':{}}

B-2 (/1pt) Ajouter au code précédent, une variable booléenne permettant la détection d'un cycle (retour à un sommet déjà visité) en renvoyant une variable booléenne dans la méthode Dijsktra().

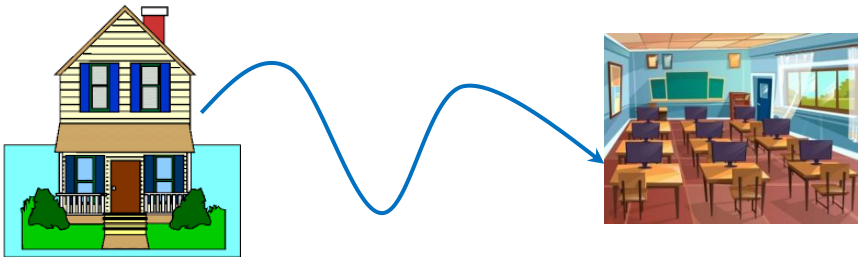
B-3 (/2pts) Dessiner ce graphe à l'aide de la fonction fournie et donner le chemin le plus court entre A et E.

B-4 (/2pts) A l'aide de la carte suivante, déterminer la liste d'adjacence des successeurs de Z vers L (une distance ne doit être reportée qu'une fois dans la liste) et appliquer l'algorithme de Djiskstra pour aller Bayonne à Lyon pour déterminer le chemin le plus court.



C (/10pts)Application à la cartographie

On désire établir l'itinéraire le plus court entre votre domicile et votre lycée. On calculera la distance de parcours pour un itinéraire en véhicule et en vélo.



Pour mener à bien cette activité pratique, vous prendrez connaissance du diaporama expliquant les différentes étapes de programmation.
Si votre domicile est à plus de 15 km de votre lycée, choisissez un lieu à proximité par exemple la mairie ou la poste de la ville où se trouve le lycée afin de réduire les temps de calculs. Récupérer dans la zone énoncée de votre classe le fichier python calculs_itineraire.py contenant toutes les fonctions de base détaillées dans l'annexe à la fin de l'énoncé. Pour faire ce TP, votre PC doit avoir accès à Internet pour récupérer les données openStreetMap et les coordonnées de géo-localisation de chaque adresse.

C-1 (/0.5pt) Déterminer en langage Python les coordonnées GPS de votre domicile et de votre lycée.

C-2 (/0.5pt) Afficher et copier le graphe de la zone géographique incluant le départ et l'arrivée. (Attention le temps de traitement peut être long selon la distance).

C-3(/0.5pt) Afficher et copier le graphe avec l'itinéraire le plus court pour les deux moyens de locomotion.

C-4 (/0.5pt) Copier le contenu de la variable route utilisé dans le programme.

C-5 (/0.5pt) Générer une page HTML permettant de visualiser le parcours en véhicule sur une carte interactive.

C-6 (/1pt) Calculer l'orientation par rapport au Nord entre votre point de départ et d'arrivée en langage Python. Vérifier la valeur trouvée sur le graphe récupéré en question **C-2** à l'aide du logiciel GIMP ou d'un rapporteur.

C-7 Détermination d'un algorithme de calcul de la distance

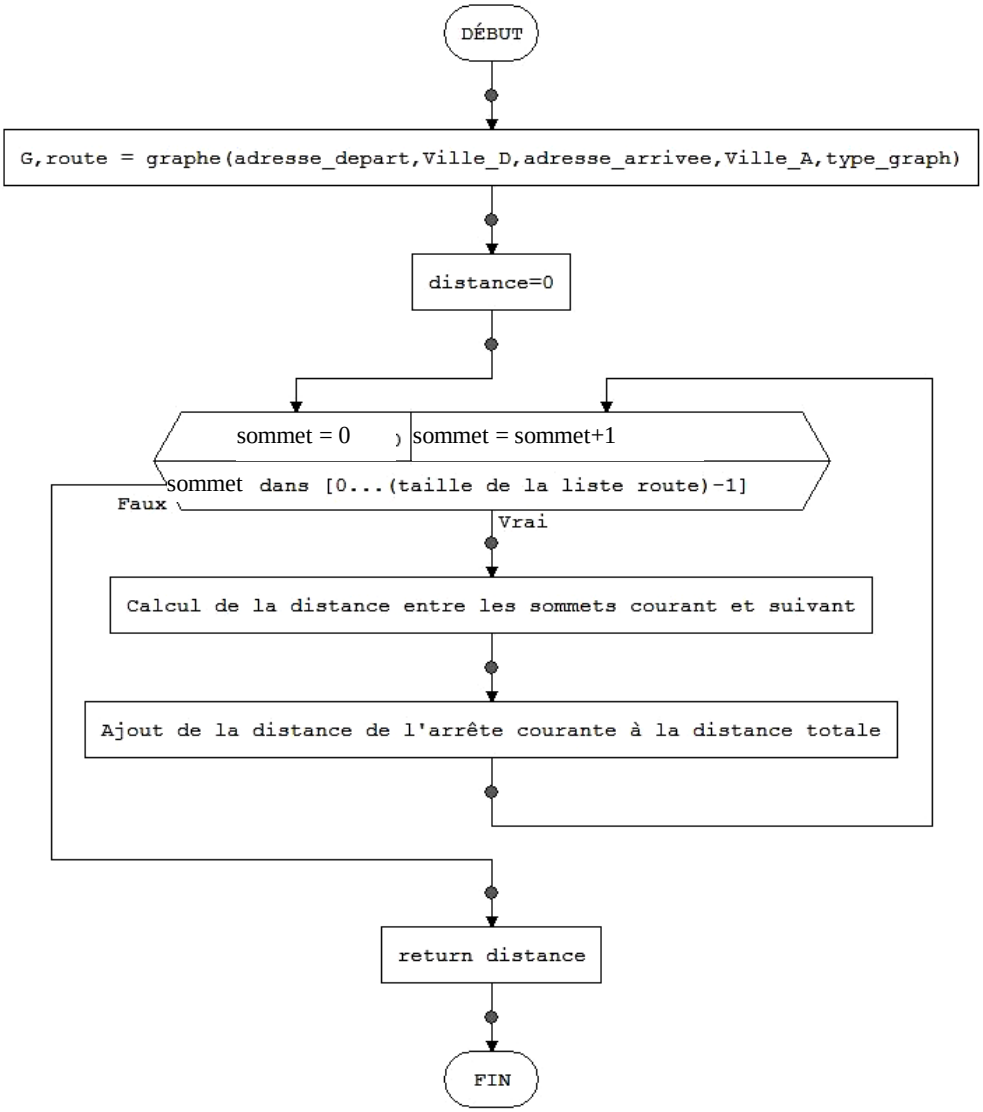
La distance totale doit être calculée en établissant la distance de chaque arrête du parcours dans le graphe G. Pour simplifier l'algorithme, on considérera les arcs comme des segments de droite.

Les identifiants des sommets sont contenus dans une liste : route=[a,b,c,d,e,f,g,h,i] avec n le nombre de points de la liste route.
Pour obtenir la latitude et la longitude d'un sommet indice k à partir de son identifiant, on utilise les lignes suivantes :
Latitude = G.node[route[k]]['y']
Longitude = G.node[route[k]]['x']

On donne le pseudo-code équivalent à l'algorithme :

```
DÉBUT
   G,route = graphe(adresse_depart,Ville_D,adresse_arrivee,Ville_A,type_graph)
   distance=0
   POUR sommet = 0 JUSQU'À (taille de la liste route)-1 INCRÉMENT 1 FAIRE
      Sommet suivant= sommet+1
      Calcul de la distance entre les sommets courant et suivant
      Ajout de la distance de l'arrête courante à la distance totale
   FINPOUR
   return distance totale
FIN
```

L'organigramme suivant décrit le fonctionnement de l'algorithme :



C-7-1(/3pts) Trouver le code python de l'algorithme permettant le calcul de la distance totale du parcours en langage Python. En observant le contenu du dictionnaire G.adj, proposer une autre solution pour déterminer la distance totale du parcours, on s'intéressera à la récupération de la distance de chaque arrête contenu dans cette variable.

C-7-2(/1pt) Programmer et tester votre fonction sur votre parcours. Donner la valeur de la distance des deux itinéraires.

C-8 Détermination du temps de parcours

C-8-1 (/1pt) Calculer le temps de parcours sachant que la limitation de vitesse est 50 km/h en ville pour les véhicules et que la vitesse moyenne d'un vélo est de 20 km/h.

C-8-2(/0.5pt) A l'aide de Google Maps, effectuer une copie d'écran d'un calcul d'itinéraires basés sur le parcours précédent en véhicule d'un graphe pondéré en temps. Comparer les itinéraires proposés et les temps théoriques et réels.

Annexe : Résumé des fonctions contenues dans le fichier calcul_itineraires.py

- Fonction déterminant les coordonnées GPS à partir d'une adresse postale, retourne un tuple (latitude, longitude) :

  coord_GPS_adresse(adresse, ville)

- Fonction de calcul de la distance entre deux points GPS, retourne un nombre décimal (float) image de la distance en mètres :

  distance_2pGPS(coord1,coord2)

- Fonction de calcul de la distance d'un arc de cercle avec les coordonnées de deux points GPS, retourne un nombre décimal (float) image de la distance en mètres :

  distance_arc_cercle(la1, lon1, la2, lon2)

- Fonction de calculs d'orientation d'une droite passant par 2 points GPS par rapport à la direction Nord, retourne un nombre décimal (float) en degrés :

  orientation(coord1,coord2)

- Fonction calcul du temps de parcours en tenant compte des vitesses maximums renseignées sur OpenStreetMap, prend en argument un graphe et une liste des points du plus court chemin et retourne un nombre décimal (float) en secondes :

  calcul_temps_parcours(G,route)

- Génération d'une carte interactive itineraire.html avec tous les détails, prend en argument un graphe et une liste des points du plus court chemin, ne retourne rien, génère une page itineraire.html à la racine du projet Python :

  graphique_html(G,route)

- Génération du graphe de la zone géographique incluant l'itinéraire fixé par les adresses de départ et d'arrivée et en fonction du type de moyen de locomotion pris en argument :

  graphe(adresse_depart,Ville_D,adresse_arrivee,Ville_A,type_graph)

- Algorithme de calcul de distance à rechercher par les élèves en activité pratique, prend en argument un graphe et une liste des points du plus court chemin et retourne un nombre décimal (float) image de la distance en mètres :

  calcul_distance(G,route)